

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1205	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	G.Saranraj	Reg.No.:	192511182
Department:		Date:	

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

I Saranraj . G / 192511182 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

ANNEXURE A – SOLUTIONS

Student Name: Saranraj.G Reg. No.: 192511182

Question 1: 2's Complement Addition and Subtraction (+45 and -23)

Problem Understanding:

The processor stores signed 8-bit integers using 2's complement representation, which can represent values from -128 to +127. We are given $A = +45$ and $B = -23$, and must analyse addition ($A+B$) and subtraction ($A-B$), including binary conversion, sign-bit alignment, and overflow detection.

Step 1 – Binary Conversion

+45 in 8-bit binary: $45 = 32+8+4+1 \rightarrow 00101101$.

For -23, first find +23 = 00010111. Invert all bits (1's complement): 11101000. Add 1 to get the 2's complement: 11101001. Therefore $-23 = 11101001$.

Step 2 – Addition ($A + B = 45 + (-23)$)

```
00101101 (+45)
+ 11101001 (-23)
-----
```

00010110 (result, with a carry-out of 1 from the MSB which is discarded)

00010110 in decimal = $16+4+2 = 22$. This matches the expected result $45 - 23 = 22$.

Step 3 – Overflow Detection for Addition

Overflow can only occur when two operands of the SAME sign are added and the result has the opposite sign. Here the operands have different signs (+45 is positive, -23 is negative), so overflow is impossible in this addition. The processor's overflow flag is set by comparing the carry into the sign bit with the carry out of the sign bit ($C_{in} \text{ XOR } C_{out}$); when they match, no overflow occurs — confirming the result 22 is correct.

Step 4 – Subtraction ($A - B = 45 - (-23)$)

Subtraction in 2's complement hardware is performed as addition of the negated subtrahend: $A - B = A + (-B)$. Since $B = -23$, $-B = +23 = 00010111$.

```
00101101 (+45)
+ 00010111 (+23)
-----
```

01000100 (result)

01000100 in decimal = $64+4 = 68$, matching $45 - (-23) = 68$.

Step 5 – Overflow Detection for Subtraction

After converting to addition, both operands (+45 and +23) are positive and the result (01000100) is also positive, so the sign did not flip and no overflow occurred. Since 68 is

well within the representable range (-128 to $+127$), the processor correctly reports a valid result with the overflow flag cleared.

Interpretation of Results:

Both operations produced arithmetically correct, valid 8-bit signed results (22 and 68 respectively) with no overflow, because in each case either the operand signs differed (ruling out overflow for the addition) or the result stayed within the ± 127 range (ruling out overflow for the subtraction). This illustrates how 2's complement lets a processor use the same adder circuit for both addition and subtraction, simply by negating the second operand when subtracting.

Question 2: 4-bit Carry Look-Ahead Adder (CLA)

Problem Understanding:

A ripple carry adder (RCA) computes each carry only after the previous stage finishes, causing carry propagation delay to grow linearly with the number of bits. A Carry Look-Ahead Adder (CLA) instead pre-computes all carries in parallel using Generate (G) and Propagate (P) signals, cutting delay significantly.

Step 1 – Generate and Propagate Signals

For each bit position i , define: Generate $G_i = A_i \cdot B_i$ (a carry is generated regardless of the incoming carry), and Propagate $P_i = A_i \oplus B_i$ (an incoming carry is passed through).

Example inputs $A = 1011$ (11) and $B = 0110$ (6), $C_0 = 0$:

$$G_0 = A_0 \cdot B_0 = 1 \cdot 0 = 0, \quad G_1 = 1 \cdot 1 = 1, \quad G_2 = 0 \cdot 1 = 0, \quad G_3 = 1 \cdot 0 = 0$$

$$P_0 = A_0 \oplus B_0 = 1, \quad P_1 = 1 \oplus 1 = 0, \quad P_2 = 0 \oplus 1 = 1, \quad P_3 = 1 \oplus 0 = 1$$

Step 2 – Look-Ahead Carry Equations

Each carry is expressed directly in terms of A , B and C_0 , without waiting for the previous carry to ripple through:

$$C_1 = G_0 + P_0 \cdot C_0$$

$$C_2 = G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0$$

$$C_3 = G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0$$

$$C_4 = G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0$$

Step 3 – Computing the Carries

$$C_1 = 0 + 1 \cdot 0 = 0$$

$$C_2 = 1 + 0 \cdot 0 + 0 \cdot 1 \cdot 0 = 1$$

$$C_3 = 0 + 1 \cdot 1 + 1 \cdot 0 \cdot 0 + 1 \cdot 0 \cdot 1 \cdot 0 = 1$$

$$C_4 = 0 + 1 \cdot 0 + 1 \cdot 1 \cdot 1 + 1 \cdot 1 \cdot 0 \cdot 0 + 1 \cdot 1 \cdot 0 \cdot 1 \cdot 0 = 1$$

Step 4 – Sum Bits and Final Result

Each sum bit is $S_i = P_i \oplus C_i$: $S_0 = P_0 \oplus C_0 = 1$, $S_1 = P_1 \oplus C_1 = 0$, $S_2 = P_2 \oplus C_2 = 0$, $S_3 = P_3 \oplus C_3 = 0$.

Result = $C_4 S_3 S_2 S_1 S_0 = 1\ 0001 = 10001$ (binary) = 17 (decimal), which correctly equals $11 + 6 = 17$.

Step 5 – Delay Comparison and Justification

In a ripple carry adder, bit i 's sum cannot be produced until C_i arrives from stage $i-1$, giving a worst-case delay proportional to n full-adder delays — roughly $2n$ gate delays for an n -bit adder (about 8 gate delays for 4 bits). In the CLA, every G_i and P_i is computed in one gate delay directly from the inputs, and every C_i is computed in at most two additional gate levels (AND-OR logic) directly from $G_0..G_i$, $P_0..P_i$ and C_0 , independent of the other carries. This gives an approximately constant delay (about 3–4 gate delays) regardless of bit width for a single CLA block.

Because carry generation no longer ripples serially through each stage, the CLA computes all carries essentially in parallel. This is why modern processors — which need wide, fast adders (32/64-bit) for ALUs — use CLA (or hierarchical CLA blocks) instead of ripple carry adders: the speed gain from avoiding serial carry propagation far outweighs the extra gate/transistor cost of the look-ahead logic.

Question 3: Booth's Algorithm for $-6 \times +5$

Problem Understanding:

Booth's algorithm multiplies signed numbers directly in 2's complement form, without a separate sign-correction step, by examining pairs of bits (Q_i, Q_{i-1}) of the multiplier and shifting an Accumulator (A), Multiplier (Q) and an extra Booth bit (Q_{-1}) together. Using 4-bit registers: Multiplicand $M = -6 = 1010$, so $-M = 0110$ (+6). Multiplier $Q = +5 = 0101$. Initial $A = 0000$, $Q_{-1} = 0$.

Step-by-Step Execution (4 cycles for 4-bit operands):

Cycle 1 — $Q_0Q_{-1} = 1,0 \rightarrow$ operation $A = A - M = A + (-M) = 0000 + 0110 = 0110$.

Arithmetic right shift of $[A \ Q \ Q_{-1}]$ gives $A = 0011$, $Q = 0010$, $Q_{-1} = 1$.

Cycle 2 — $Q_0Q_{-1} = 0,1 \rightarrow$ operation $A = A + M = 0011 + 1010 = 1101$. Arithmetic right shift gives $A = 1110$, $Q = 1001$, $Q_{-1} = 0$.

Cycle 3 — $Q_0Q_{-1} = 1,0 \rightarrow$ operation $A = A - M = A + (-M) = 1110 + 0110 = 0100$.

Arithmetic right shift gives $A = 0010$, $Q = 0100$, $Q_{-1} = 1$.

Cycle 4 — $Q_0Q_{-1} = 0,1 \rightarrow$ operation $A = A + M = 0010 + 1010 = 1100$. Arithmetic right shift gives $A = 1110$, $Q = 0010$, $Q_{-1} = 0$.

Step – Final Result

After 4 cycles, the product is the concatenation of A and Q: 1110 0010 (8 bits). As an 8-bit 2's complement number this equals -30 , which correctly matches $(-6) \times (+5) = -30$.

Role of the Registers and Efficiency

The Accumulator (A) holds the running partial sum, the Multiplier register (Q) is progressively shifted out and replaced by the growing product's lower bits, and the extra Booth bit (Q_{-1}) remembers the previously examined multiplier bit so the pair (Q_0, Q_{-1}) can classify each step as "start of a run of 1s" ($10 \rightarrow$ subtract), "end of a run of 1s" ($01 \rightarrow$ add), or "inside a run of 0s/1s" (00 or $11 \rightarrow$ no arithmetic operation, shift only). By replacing long runs of consecutive 1s in the multiplier with a single subtraction at the start and a single addition at the end, Booth's algorithm reduces the number of add/subtract operations compared to naive shift-and-add multiplication (which would add for every 1 bit), and it handles negative operands correctly without any separate sign-magnitude conversion, since all arithmetic is done directly on 2's complement values.

Question 4: Restoring vs Non-Restoring Division ($13 \div 3$)

Problem Understanding:

Both algorithms divide Dividend $Q = 13$ (1101) by Divisor $M = 3$ (0011) using 4-bit registers, an Accumulator A (initially 0000) that holds the partial remainder, and repeated shift-and-subtract cycles.

Restoring Division – Step by Step

Rule: shift (A, Q) left by 1, compute $A = A - M$; if the result is negative, RESTORE A to its value before the subtraction and set the new quotient bit $Q_0 = 0$; otherwise keep the subtracted A and set $Q_0 = 1$.

Cycle 1: shift $\rightarrow A=0001$, $Q=1010$; $A-M = 0001-0011 = \text{negative}$ $\rightarrow$ restore $A=0001$, $Q_0=0 \rightarrow Q=1010$.

Cycle 2: shift $\rightarrow A=0011$, $Q=0100$; $A-M = 0011-0011 = 0000$ (non-negative) $\rightarrow$ keep $A=0000$, $Q_0=1 \rightarrow Q=0101$.

Cycle 3: shift $\rightarrow A=0000$, $Q=1010$; $A-M = 0000-0011 = \text{negative}$ $\rightarrow$ restore $A=0000$, $Q_0=0 \rightarrow Q=1010$.

Cycle 4: shift $\rightarrow A=0001$, $Q=0100$; $A-M = 0001-0011 = \text{negative}$ $\rightarrow$ restore $A=0001$, $Q_0=0 \rightarrow Q=0100$.

Final result: Remainder = $A = 0001 = 1$, Quotient = $Q = 0100 = 4$. Check: $3 \times 4 + 1 = 13 \checkmark$.

Non-Restoring Division – Step by Step

Rule: shift (A, Q) left by 1; if the previous A was non-negative, subtract M ($A = A - M$), otherwise add M ($A = A + M$); set $Q_0 = 1$ if the new A is non-negative, else $Q_0 = 0$. No restoring step is done mid-way — only a single correction at the very end if the final A is negative.

Cycle 1: shift $\rightarrow A=0001$; subtract (prev $A \geq 0$): $A=0001-0011=1110$ (negative) $\rightarrow Q_0=0 \rightarrow Q=1010$.

Cycle 2: shift $\rightarrow A=1101$; add (prev $A < 0$): $A=1101+0011=0000$ (non-negative) $\rightarrow Q_0=1 \rightarrow Q=0101$.

Cycle 3: shift $\rightarrow A=0000$; subtract (prev $A \geq 0$): $A=0000-0011=1101$ (negative) $\rightarrow Q_0=0 \rightarrow Q=1010$.

Cycle 4: shift $\rightarrow A=1011$; add (prev $A < 0$): $A=1011+0011=1110$ (negative) $\rightarrow Q_0=0 \rightarrow Q=0100$.

Final correction: since the last A (1110) is negative, add M once: $A = 1110+0011 = 0001$.

Final result: Remainder = $A = 0001 = 1$, Quotient = $Q = 0100 = 4$, identical to the restoring method — $3 \times 4 + 1 = 13 \checkmark$.

Comparison and Justification

Both methods take the same number of cycles (equal to the register width) and produce identical results. However, restoring division may need an extra "add back" (restore) operation in every cycle where the trial subtraction goes negative, so in the worst case it performs up to $2n$ arithmetic operations for n bits. Non-restoring division always performs

exactly one add or subtract per cycle (n operations total) plus at most one final correction, roughly halving the number of arithmetic operations. Because it avoids the conditional restore step inside the loop, non-restoring division has a simpler, more uniform, and more pipeline-friendly hardware datapath, which is why it is generally preferred over restoring division in modern processor design.

Normalization ensures the mantissa always has exactly one non-zero leading bit before the binary point, which maximises precision for a fixed mantissa width and keeps the encoding unique. Rounding (typically round-to-nearest-even) becomes necessary whenever the aligned/added mantissa needs more bits than the format's mantissa field provides — the extra low-order bits (guard, round and sticky bits) are used to decide whether to round up, introducing a small rounding error. Precision loss chiefly comes from this rounding step and

from the alignment shift, where low-order bits of the smaller-exponent operand can be shifted out entirely; this is most severe when adding numbers of very different magnitudes, a well-known limitation of finite-precision floating-point arithmetic.